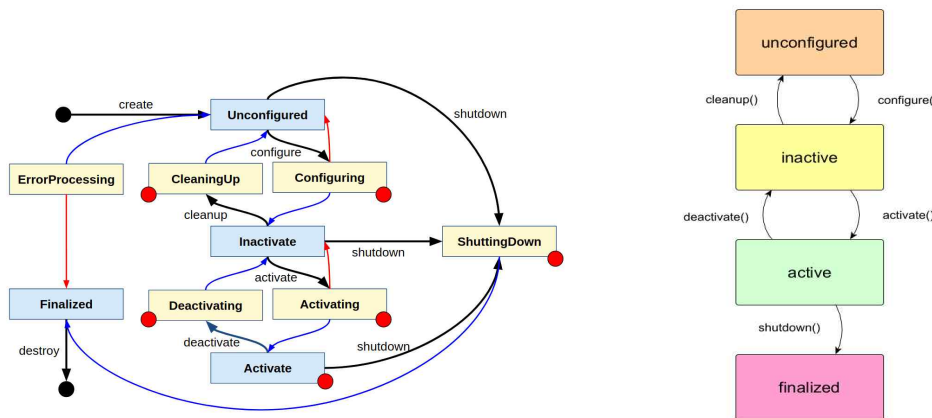


ROS2 4일차 과제1 보고서, 김도훈, 2026406041

<목차>

1. Lifecycle 이란?
2. Lifecycle을 적용해야 하는 이유
3. Lifecycle을 구현하기에 적합한 패키지
4. 각 노드에 적합한 QoS설정 및 근거

1.Lifecycle이란?



ROS 2에서 로봇 프로그램을 만들 때는 여러 개의 Node를 사용한다. Node는 ROS 2에서 하나의 기능을 담당하는 프로그램의 기본 단위라고 볼 수 있다. 예를 들어 자율 이동 로봇을 만든다고 하면 LiDAR 센서의 데이터를 보내는 Node, 현재 로봇의 위치를 계산하는 Node, 목적지까지 이동 경로를 만드는 Node, 모터를 제어하는 Node 등이 각각 존재할 수 있다. 일반적인 ROS2 Node는 프로그램을 실행하면 바로 동작을 시작하도록 구현하는 경우가 많다. 예를 들어 Publisher Node를 실행하면 바로 메시지를 보내고 Subscriber Node를 실행하면 바로 메시지를 받기 시작한다.

하지만 실제 로봇에서는 프로그램이 실행되었다고 해서 바로 로봇을 동작시키는 것이 항상 좋은 방법은 아니다. 예를 들어 모터를 제어하는 Node가 있다고 생각해 보자. 프로그램을 실행하자마자 모터를 움직이는 것보다 먼저 모터와 정상적으로 통신할 수 있는지 확인하고 필요한 값을 불러온 다음 모든 세팅이 완료되었을 때 모터 제어를 시작하는 것이 더 안전하다. 이처럼 Node의 상태를 단계별로 관리하기 위해 ROS2에서는 Lifecycle이라고 한다. 쉽게 말해 Lifecycle은 한 로봇의 생명 주기와 같다.

2. Lifecycle을 적용해야 하는 이유

간단한 Publisher와 Subscriber 예제에서는 Lifecycle이 없어도 큰 문제가 없을 수 있다. Publisher가 데이터를 보내고 Subscriber가 데이터를 받는 것만 확인한다면 프로그램을 실행하자마자 동작해도 되기 때문이다. 하지만 실제 로봇은 하나의 Node만 가지고 동작하지 않는다. 예를 들어 이동 로봇은 다음과 같은 여러 기능을 사용할 수 있다. LiDAR -> 위치 추정 -> 경로 생성 -> 로봇 제어 -> Motor 각 기능을 ROS 2 Node로 나누면 다음과 같이 생각할 수 있다. LiDAR Node -> Localization Node -> Planner Node -> Controller Node -> Motor

이때 각각의 Node는 서로 완전히 독립적인 것이 아니라 앞에 있는 Node에서 만들어진 데이터를 사용해서 다음 Node가 동작하는 경우가 많다. 예를 들어 Localization Node는 LiDAR 데이터를 이용해 현재 로봇의 위치를 계산한다. Planner Node는 현재 로봇의 위치를 알아야 목적지까지의 이동 경로를 계산할 수 있다. Controller Node는 Planner가 만든 경로를 이용하여 실제 로봇의 이동 방향과 속도를 결정할 수 있다. 이러한 과정에서 LiDAR가 준비되지 않았거나 Localization이 준비되지 않았거나 Planner가 준비되지 않았는데 Controller가 동작한다면 어떻게 될까? 로봇이 위치나 경로도 제대로 계산하지 못했는데 Controller가 먼저 동작하는 것은 좋은 방법이 아니다. 이러한 문제를 해결하기 위해서 Lifecycle이 필요한 것이다.

3. Lifecycle을 구현하기에 적합한 패키지

Lifecycle은 실제 로봇에서 사용하는 센서나 모터와 같이 프로그램 실행 전에 준비 과정이 필요한 패키지에 적용하기 적합하다고 생각한다. 예를 들어 이동 로봇에서 사용하는 LiDAR 센서 패키지를 생각할 수 있다. LiDAR는 레이저를 이용하여 로봇 주변에 있는 물체까지의 거리를 측정하고 측정한 데이터를 Topic으로 전달하는 센서이다. 일반적인 Node로 LiDAR 프로그램을 구현하면 Node가 실행되면서 바로 센서 데이터를 전송하도록 만들 수 있다. 하지만 실제 센서를 사용할 때는 먼저 센서가 정상적으로 연결되어 있는지 확인하고 필요한 설정값을 적용한 후 데이터를 전송하는 것이 더 안정적이다. 이러한 과정에 Lifecycle을 적용하면 Node가 처음 실행되었을 때는 Unconfigured 상태로 두고, Configure 과정에서 LiDAR 센서의 연결을 확인하고 Publisher와 필요한 변수들을 준비할 수 있다. Configure가 정상적으로 끝나면 Inactive 상태가 된다. 이때는 센서를 사용할 준비는 완료되었지만 아직 데이터를 보내지 않는다. 이후 Activate를 수행하면 Active 상태가 되고 실제 LiDAR 데이터를 측정하여 Topic으로 전달하도록 만들 수 있다. 반대로 센서 데이터를 잠시 사용하지 않아야 할 경우 Deactivate를 수행하여 다시 Inactive 상태로 만들 수 있다. 센서 설정을 처음부터 다시 해야 하는 경우에는 Cleanup을 수행하여 준비했던 자원을 정리하고 Unconfigured 상태로 돌아갈 수 있다. 따라서 LiDAR와 같은 센서 패키지는 센서 연결과 설정을 하는 단계와 실제 센서 데이터를 사용하는 단계를 나눌 수 있기 때문에 Lifecycle을 적용하기 적합하다고 생각한다.

4. 각 노드에 적합한 QoS 설정 및 근거

ROS2에서 QoS는 Publisher와 Subscriber 사이에서 데이터를 어떤 방식으로 전달할지를 정하는 설정이다. LiDAR는 주변 환경을 계속 측정하면서 짧은 주기로 많은 데이터를 보내는 센서이기 때문에 모든 데이터를 완벽하게 저장하고 전달하는 것보다 현재의 최신 데이터를 빠르게 전달하는 것이 중요하다. 그래서 History는 Keep Last를 사용하고 Depth는 5로 설정해야한다. LiDAR 데이터는 계속 새롭게 생성되기 때문에 오래된 데이터를 많이 저장할 필요가 없다. 최근 데이터 몇 개만 저장해도 현재 주변 환경을 확인하는 데 충분하고 오래된 데이터가 계속 쌓이면 오히려 불필요한 메모리 사용이 증가할 수 있다. 따라서 최근 5개의 데이터만 유지하도록 설정하는 것이 적합하다.

그리고 Reliability는 Best Effort를 사용해야한다. LiDAR는 매우 빠른 주기로 데이터를 반복해서 보내기 때문에 하나의 데이터가 손실되더라도 바로 다음 데이터가 들어온다. 예를 들어 한 번의 거리 측정값이 손실되더라도 다음 측정값을 통해 현재 장애물의 위치를 다시 확인할 수 있다. 따라서 손실된 데이터를 다시 보내기 위해 기다리는 것보다 새로운 데이터를 빠르게 받아 처리하는 것이 더 중요하다고 생각한다. Durability는 Volatile로 설정하였다. LiDAR의 과거 데이터는 현재 로봇 주변 환경을 나타내지 않을 수 있기 때문에 Subscriber가 늦게 실행되었을 때 이전 데이터를 다시 받을 필요가 없다. Subscriber가 실행된 시점부터 새롭게 들어오는 최신 센서 데이터만 받아도 충분하다.

Subscriber 역시 Publisher와 같은 Keep Last, Depth 5, Best Effort, Volatile을 사용하도록 설정해야한다. Publisher와 Subscriber의 QoS는 서로 호환되어야 정상적인 통신이 가능하고 같은 센서 데이터를 주고받는 구조에서는 동일한 설정을 사용하는 것이 이해하기 쉽고 통신 문제를 줄이는 데 도움이 된다. 따라서 LiDAR Publisher와 Subscriber에는 Keep Last, Depth 5, Best Effort, Volatile 설정이 적합하다고 판단했다. 이는 LiDAR 데이터가 빠르게 반복해서 생성되고 과거 데이터보다 최신 데이터가 더 중요하다는 특징을 반영한 판단이다.